

COMPUTER ARCHITECTURE ASSESSMENT TOOL

CO3 AT1

NAME:SIVARANGJENII B

REG.NO:192521375

SIMATS ENGINEERING Department of Computer Science and Engineering ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyse how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyse a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

1. Register Transfer and ALU Operations:

Load me at

1 LDA 8050

2 MOV B,A

3 LDA 8051

4 MOV C,A

5 MOV D,B

6 MOV A,B

7 ADD C

8 STA 8052

9 MOV A,B

10 SUB C

11 STA 8053

12 MOV A,B

13 ANA C

14 STA 8054

15 MOV A,B

16 ORA C

17 STA 8055

18 HLT

DataStackKeyPadMemor

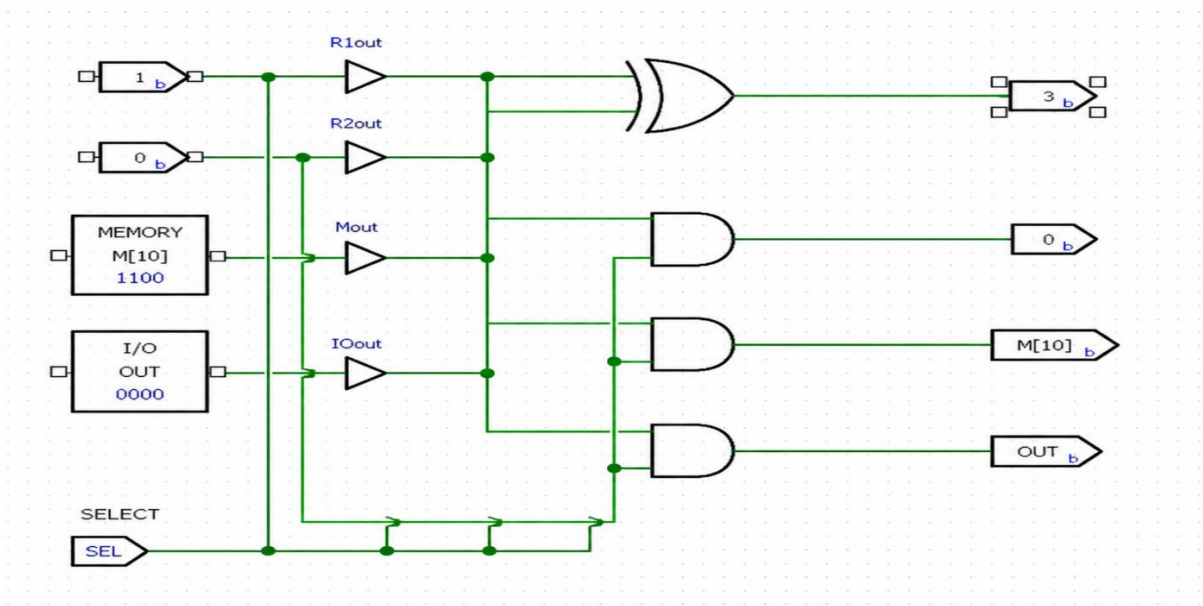
Start 8050

Address (Hex)	Address	Data
1F72	8050	5
1F73	8051	3
1F74	8052	8
1F75	8053	2
1F76	8054	1
1F77	8055	7
1F78	8056	0
1F79	8057	0
1F7A	8058	0
1F7B	8059	0
1F7C	8060	0
1F7D	8061	0
1F7E	8062	0
1F7F	8063	0

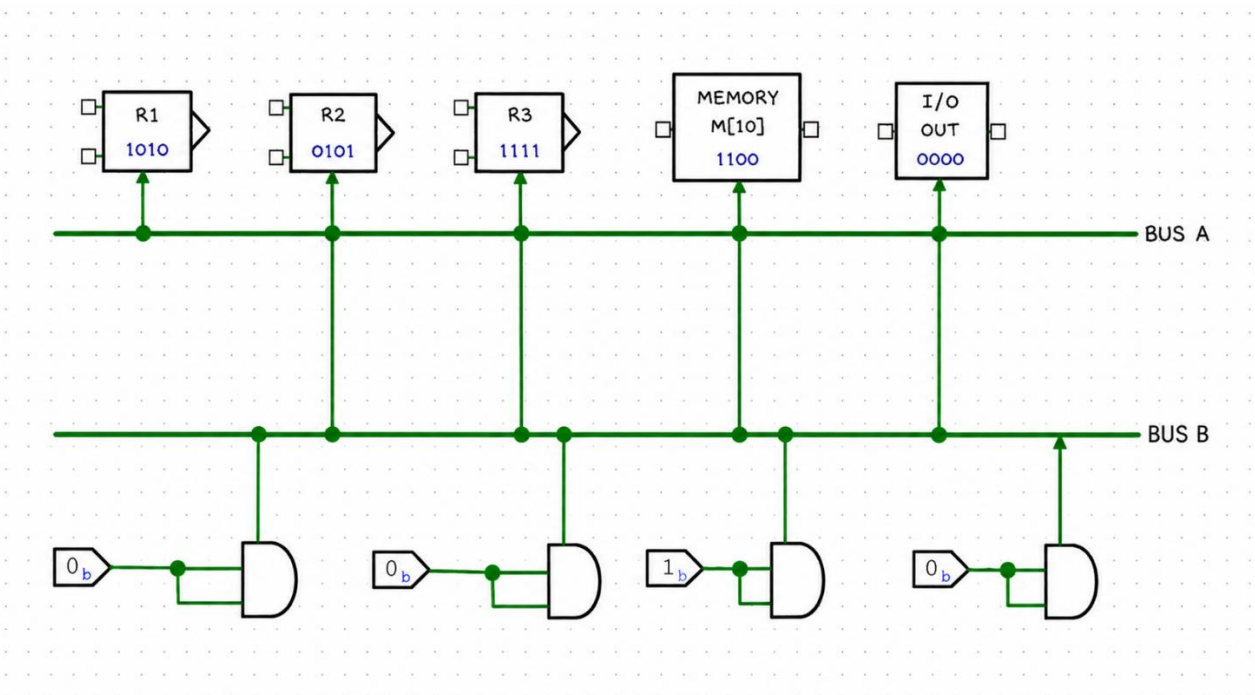
Line No	Assembler Message
0	Program assembled successfully

This experiment demonstrates how data is transferred between processor registers and how the ALU (Arithmetic Logic Unit) performs basic operations such as addition, subtraction, AND, and OR on the data. It helps understand the sequence of data movement and processing inside a CPU.

2. SINGLE BUS ORGANIZATION:



MULTI BUS ORGANIZATION:



A single-bus and multi-bus organization demonstrates data transfer between registers, memory, I/O, and the ALU, and compares their speed, efficiency, and parallel data-transfer capability.

3.

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

[1] ✓ 0s # 5-Stage Instruction Pipeline Simulation

instructions = [
    "I1: ADD R1,R2,R3",
    "I2: SUB R4,R1,R5",
    "I3: AND R6,R4,R7",
    "I4: OR R8,R6,R9",
    "I5: ADD R10,R8,R11"
]

stages = ["IF", "ID", "EX", "MEM", "WB"]

# Create pipeline table
print("5-STAGE INSTRUCTION PIPELINE")
print("=" * 60)

for cycle in range(1, 10):
    print(f"Cycle {cycle}:", end=" ")

    for i in range(len(instructions)):
        stage_index = cycle - i - 1

        if 0 <= stage_index < 5:
            print(f"{instructions[i]} -> {stages[stage_index]}", end=" | ")

    print()
```

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

[1] ✓ 0s print()

# Performance calculation
n = len(instructions)
s = len(stages)

non_pipelined_cycles = n * s
pipelined_cycles = n + s - 1

speedup = non_pipelined_cycles / pipelined_cycles
throughput = n / pipelined_cycles

print("\n" + "=" * 60)
print("PERFORMANCE ANALYSIS")
print("=" * 60)

print("Number of Instructions :", n)
print("Number of Stages      :", s)
print("Non-Pipelined Cycles   :", non_pipelined_cycles)
print("Pipelined Cycles        :", pipelined_cycles)
print("Speedup                  :", round(speedup, 2))
print("Throughput               :", round(throughput, 2), "instructions/cycle")

print("\nConclusion:")
print("Pipelining reduces execution time and increases instruction throughput.")
```

OUTPUT

```
Untitled4.ipynb ☆
File Edit View Insert Runtime Tools Help
mands + Code + Text ▶ Run all

[1] print("Throughput          :", round(throughput, 2), "instructions/cycle")
✓ Os print("\nConclusion:")
     print("Pipelining reduces execution time and increases instruction throughput.")

... 5-STAGE INSTRUCTION PIPELINE
=====
Cycle 1: I1: ADD R1,R2,R3 -> IF |
Cycle 2: I1: ADD R1,R2,R3 -> ID | I2: SUB R4,R1,R5 -> IF |
Cycle 3: I1: ADD R1,R2,R3 -> EX | I2: SUB R4,R1,R5 -> ID | I3: AND R6,R4,R7 -> IF |
Cycle 4: I1: ADD R1,R2,R3 -> MEM | I2: SUB R4,R1,R5 -> EX | I3: AND R6,R4,R7 -> ID | I4: OR R8,R6,R9 -> IF |
Cycle 5: I1: ADD R1,R2,R3 -> WB | I2: SUB R4,R1,R5 -> MEM | I3: AND R6,R4,R7 -> EX | I4: OR R8,R6,R9 -> ID | I5: ADD R10,R8,R11 -> IF |
Cycle 6: I2: SUB R4,R1,R5 -> WB | I3: AND R6,R4,R7 -> MEM | I4: OR R8,R6,R9 -> EX | I5: ADD R10,R8,R11 -> ID |
Cycle 7: I3: AND R6,R4,R7 -> WB | I4: OR R8,R6,R9 -> MEM | I5: ADD R10,R8,R11 -> EX |
Cycle 8: I4: OR R8,R6,R9 -> WB | I5: ADD R10,R8,R11 -> MEM |
Cycle 9: I5: ADD R10,R8,R11 -> WB |

=====
PERFORMANCE ANALYSIS
=====
Number of Instructions : 5
Number of Stages       : 5
Non-Pipelined Cycles   : 25
Pipelined Cycles       : 9
Speedup                 : 2.78
Throughput              : 0.56 instructions/cycle

Conclusion:
Pipelining reduces execution time and increases instruction throughput.
```

A **5-stage instruction pipeline** divides instruction execution into Fetch, Decode, Execute, Memory, and Write-back stages, allowing multiple instructions to execute simultaneously and improving **processor speed, throughput, and efficiency**.

4.

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

[2] ✓ Os ▶ # Pipeline Hazards Simulation
# Data Hazard, Control Hazard, Structural Hazard

print("=" * 60)
print("PIPELINE HAZARD SIMULATION")
print("=" * 60)

# -----
# 1. DATA HAZARD
# -----

print("\n1. DATA HAZARD")
print("-" * 40)

print("I1: ADD R1, R2, R3")
print("I2: SUB R4, R1, R5")

print("\nProblem:")
print("I2 needs R1 produced by I1.")

cycles_without_forwarding = 7
cycles_with_forwarding = 6

print("\nWithout Data Forwarding:")
print("Stall inserted")
print("Cycles =", cycles_without_forwarding)
```

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

[2] ✓ Os ▶ print("\nStall inserted",
print("Cycles =", cycles_without_forwarding)

print("\nWith Data Forwarding:")
print("Result forwarded from EX stage")
print("Cycles =", cycles_with_forwarding)

print("Cycles saved =",
      cycles_without_forwarding - cycles_with_forwarding)

# -----
# 2. CONTROL HAZARD
# -----

print("\n2. CONTROL HAZARD")
print("-" * 40)

print("I1: BEQ R1, R2, LABEL")
print("I2: ADD R3, R4, R5")

print("\nProblem:")
print("Processor does not immediately know whether branch is taken.")

# Branch prediction
prediction = "TAKEN"
actual = "TAKEN"
```


[2]

✓ 0s



```
print("\nBranch Prediction =", prediction)
print("Actual Result      =", actual)
```

```
if prediction == actual:
    print("Prediction CORRECT")
    print("No pipeline flush required")
else:
    print("Prediction WRONG")
    print("Pipeline must be flushed")
```

```
# -----
# 3. STRUCTURAL HAZARD
# -----
```

```
print("\n3. STRUCTURAL HAZARD")
print("-" * 40)
```

```
print("Instruction Fetch and Memory Access")
print("both require the same memory resource.")
```

```
cycles_structural_without_solution = 7
cycles_structural_with_solution = 6
```

```
print("\nWithout Additional Hardware:")
print("Stall required")
```



```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

[2] ✓ 0s ▶ print("Stall required")
print("Cycles =", cycles_structural_without_solution)

print("\nWith Separate Instruction/Data Memory:")
print("No structural conflict")
print("Cycles =", cycles_structural_with_solution)

# -----
# 4. FINAL PERFORMANCE COMPARISON
# -----

print("\n" + "=" * 60)
print("FINAL PERFORMANCE COMPARISON")
print("=" * 60)

print("\nData Hazard:")
print("Without Forwarding =", cycles_without_forwarding, "cycles")
print("With Forwarding      =", cycles_with_forwarding, "cycles")

print("\nStructural Hazard:")
print("Without Solution =", cycles_structural_without_solution, "cycles")
print("With Solution     =", cycles_structural_with_solution, "cycles")

print("\nControl Hazard:")
print("Branch Prediction =", prediction)
print("Prediction Result = CORRECT")
```

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

[2] ✓ 0s ▶ print("\n" + "=" * 60)
print("CONCLUSION")
print("=" * 60)

print("Pipeline hazards can reduce processor performance.")
print("Data forwarding reduces data-hazard delays.")
print("Branch prediction reduces control-hazard delays.")
print("Separate hardware resources reduce structural hazards.")
print("Therefore, hazard-handling techniques improve pipeline performance.")
```

OUTPUT:

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text | ▶ Run all

... PIPELINE HAZARD SIMULATION
=====

1. DATA HAZARD
-----
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5

Problem:
I2 needs R1 produced by I1.

Without Data Forwarding:
Stall inserted
Cycles = 7

With Data Forwarding:
Result forwarded from EX stage
Cycles = 6
Cycles saved = 1

2. CONTROL HAZARD
-----
I1: BEQ R1, R2, LABEL
I2: ADD R3, R4, R5

Problem:
Processor does not immediately know whether branch is taken.

Branch Prediction = TAKEN
Actual Result      = TAKEN
Prediction CORRECT
No pipeline flush required

3. STRUCTURAL HAZARD
-----
Instruction Fetch and Memory Access
both require the same memory resource.

Without Additional Hardware:
Stall required
Cycles = 7

With Separate Instruction/Data Memory:
No structural conflict
Cycles = 6
```

```
=====
FINAL PERFORMANCE COMPARISON
=====
```

Data Hazard:

Without Forwarding = 7 cycles

With Forwarding = 6 cycles

Structural Hazard:

Without Solution = 7 cycles

With Solution = 6 cycles

Control Hazard:

Branch Prediction = TAKEN

Prediction Result = CORRECT

```
=====
CONCLUSION
=====
```

Pipeline hazards can reduce processor performance.

Data forwarding reduces data-hazard delays.

Branch prediction reduces control-hazard delays.

Separate hardware resources reduce structural hazards.

Therefore, hazard-handling techniques improve pipeline performance.

Pipeline hazards are conditions that delay instruction execution in a pipeline; **stalling, data forwarding, and branch prediction** are techniques used to reduce these delays and improve processor performance.

5.

```
Untitled4.ipynb ☆ Saving...
File Edit View Insert Runtime Tools Help
Commands + Code + Text | ▶ Run all

[3] ✓ Os
# Advanced Processor Execution Simulation
# Superscalar, Out-of-Order, Speculative Execution,
# Hyper-Threading and SMT

print("=" * 70)
print("ADVANCED PROCESSOR EXECUTION SIMULATION")
print("=" * 70)

# -----
# 1. SUPERSCALAR EXECUTION
# -----

print("\n1. SUPERSCALAR EXECUTION")
print("-" * 50)

instructions = ["I1", "I2", "I3", "I4", "I5", "I6"]

print("Normal Processor:")
for i, instruction in enumerate(instructions, 1):
    print("Cycle", i, ":", instruction)

print("\nSuperscalar Processor (2 instructions per cycle):")

cycle = 1
for i in range(0, len(instructions), 2):
    print("Cycle", cycle, ":", instructions[i:i+2])
    cycle += 1

normal_cycles = 6
superscalar_cycles = 3

print("\nNormal Cycles =", normal_cycles)
print("Superscalar Cycles =", superscalar_cycles)

# -----
# 2. OUT-OF-ORDER EXECUTION
# -----

print("\n2. OUT-OF-ORDER EXECUTION")
print("-" * 50)

print("Instructions:")
print("I1 -> Waiting for data")
print("I2 -> Ready")
print("I3 -> Ready")

print("\nExecution Order:")
print("Cycle 1 -> I2")
print("Cycle 2 -> I3")
print("Cycle 3 -> I1")
```

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code ▾ + Text | ▶ Run all ▾

[3]
✓ 0s ▶ # -----
# 7. FINAL SUMMARY
# -----

print("\n" + "=" * 70)
print("FINAL SUMMARY")
print("=" * 70)

techniques = {
    "Superscalar Execution":
        "Executes multiple instructions per cycle",

    "Out-of-Order Execution":
        "Executes ready instructions before waiting instructions",

    "Speculative Execution":
        "Executes instructions based on branch prediction",

    "Hyper-Threading":
        "Allows multiple logical threads on one physical core",

    "SMT":
        "Executes instructions from multiple threads simultaneously"
}

for technique, description in techniques.items():
```

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code ▾ + Text | ▶ Run all ▾

[3]
✓ 0s ▶ "Speculative Execution":
        "Executes instructions based on branch prediction",

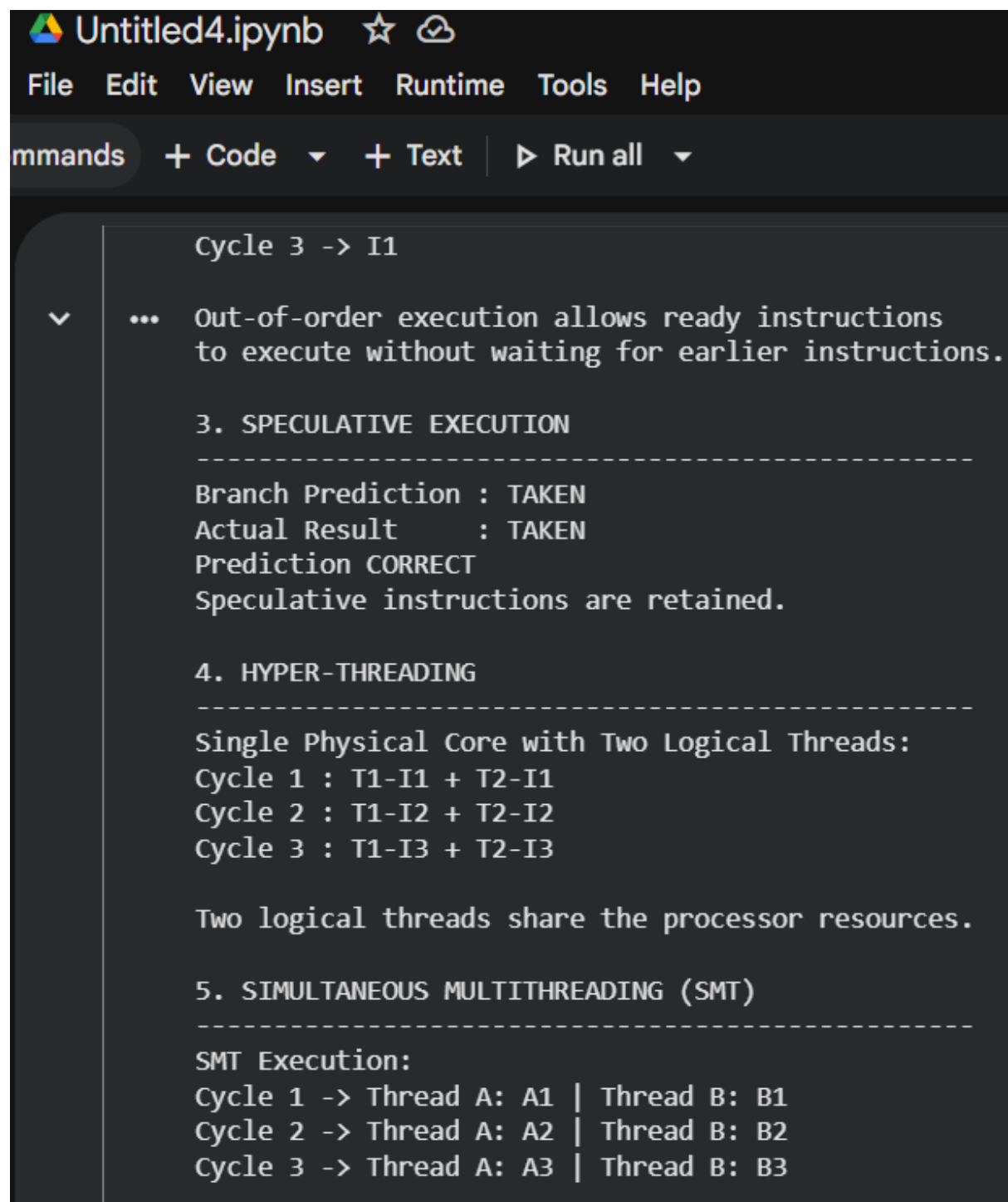
        "Hyper-Threading":
            "Allows multiple logical threads on one physical core",

        "SMT":
            "Executes instructions from multiple threads simultaneously"
    }

    for technique, description in techniques.items():
        print("\n" + technique)
        print("->", description)

    print("\nConclusion:")
    print("Advanced processor techniques increase parallelism,")
    print("improve processor resource utilization, and increase performance.")
```

OUTPUT:



```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code ▾ + Text | ▶ Run all ▾

Cycle 3 -> I1

... Out-of-order execution allows ready instructions
to execute without waiting for earlier instructions.

3. SPECULATIVE EXECUTION
-----
Branch Prediction : TAKEN
Actual Result      : TAKEN
Prediction CORRECT
Speculative instructions are retained.

4. HYPER-THREADING
-----
Single Physical Core with Two Logical Threads:
Cycle 1 : T1-I1 + T2-I1
Cycle 2 : T1-I2 + T2-I2
Cycle 3 : T1-I3 + T2-I3

Two logical threads share the processor resources.

5. SIMULTANEOUS MULTITHREADING (SMT)
-----
SMT Execution:
Cycle 1 -> Thread A: A1 | Thread B: B1
Cycle 2 -> Thread A: A2 | Thread B: B2
Cycle 3 -> Thread A: A3 | Thread B: B3
```

```
Untitled4.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help

Commands + Code ▾ + Text | ▶ Run all ▾

=====
PERFORMANCE COMPARISON
=====

Normal Execution      : 6 cycles
Superscalar Execution : 3 cycles
Superscalar Speedup   : 2.0

=====
FINAL SUMMARY
=====

Superscalar Execution
-> Executes multiple instructions per cycle

Out-of-Order Execution
-> Executes ready instructions before waiting instructions

Speculative Execution
-> Executes instructions based on branch prediction

Hyper-Threading
-> Allows multiple logical threads on one physical core

SMT
-> Executes instructions from multiple threads simultaneously

Conclusion:
Advanced processor techniques increase parallelism,
improve processor resource utilization, and increase performance.
```

Superscalar and SMT processors improve performance by executing multiple instructions or threads in parallel using techniques such as **out-of-order execution, speculative execution, and simultaneous multithreading**.